

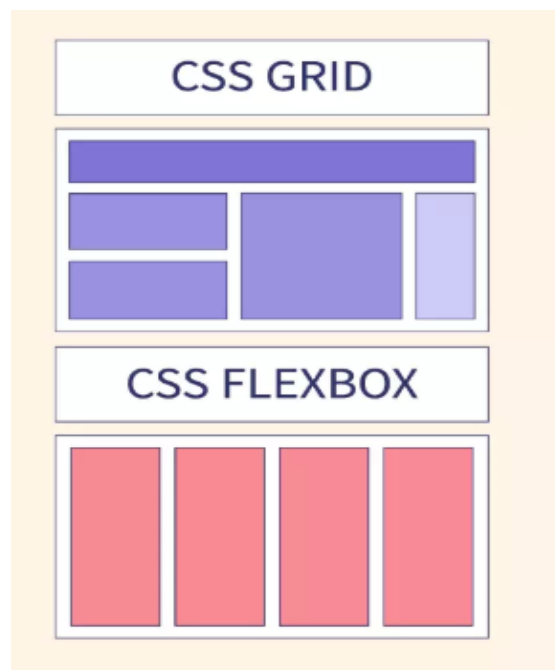
Apuntes de Flexbox y css grid

1. Diseño web avanzado con CSS Grid y Flexbox

Flexbox y Grid sirven para organizar el layout de una página. No hacen exactamente lo mismo, pero se pueden combinar.

Para recordar

- **display: flex:** activa Flexbox.
- **display: grid:** activa CSS Grid.
- **gap:** deja espacio entre los elementos.



2. Conceptos fundamentales de CSS: display y posicionamiento

display cambia la forma en que se comporta un elemento y position controla cómo se coloca dentro de la página.

Para recordar

- **display: block:** ocupa una línea completa.
- **display: inline:** se mantiene dentro de la misma línea.
- **position:** define cómo se posiciona un elemento.
- **top / right / bottom / left:** permiten mover un elemento cuando el tipo de position lo permite.

3. Display: block, inline e inline-block

El navegador normalmente acomoda los elementos siguiendo el flujo normal. Por eso es importante saber cómo funcionan los diferentes tipos de display.

Para recordar

- **block:** se comporta como un bloque.
- **inline:** se comporta dentro de la línea de texto.
- **inline-block:** queda en línea pero permite controlar mejor sus dimensiones.

4. Formato de contexto de bloque (BFC)

Un BFC crea un contexto de formato de bloque. Puede ayudar a separar comportamientos del layout y evitar algunos problemas con float y márgenes.

Para recordar

- **display: flow-root:** crea un nuevo BFC.
- **overflow:** algunos valores también pueden crear un nuevo contexto.

5. Posicionamiento y contexto de apilamiento

position no solamente sirve para mover elementos. También puede afectar la forma en que se apilan unos sobre otros.

Para recordar

- **relative:** mantiene el espacio original y permite mover el elemento.
- **absolute:** sale del flujo normal y se posiciona respecto a un elemento de referencia.

- **fixed:** queda fijo respecto a la ventana.
- **sticky:** puede quedarse pegado durante el scroll.
- **z-index:** controla qué elemento aparece por encima de otro.

6. Diferencias entre CSS Grid y Flexbox

Una forma fácil de recordarlo:

Flexbox → una dimensión.

Grid → dos dimensiones.

Flexbox funciona muy bien para organizar elementos en una fila o columna. Grid está pensado para trabajar con filas y columnas al mismo tiempo.

Para recordar

- **flex-direction:** define la dirección de Flexbox.
- **justify-content:** alinea en el eje principal.
- **align-items:** alinea en el eje transversal.
- **grid-template-columns:** define las columnas.
- **grid-template-rows:** define las filas.

7. Alineación en Flexbox y Grid

Flexbox y Grid tienen varias propiedades parecidas para distribuir y alinear elementos. Lo importante es aplicarlas al contenedor correcto.

Para recordar

- **gap:** separa los elementos.
- **align-items:** alinea los elementos.
- **justify-content:** distribuye el espacio disponible.

8. Uso combinado de Flexbox y CSS Grid

No es necesario escoger solamente uno. Se pueden usar juntos.

Por ejemplo, Grid puede servir para organizar la estructura general de una página y Flexbox para organizar los elementos dentro de una tarjeta, menú o botón.

Para recordar

- **Grid:** útil para estructuras con filas y columnas.
- **Flexbox:** útil para alinear elementos dentro de componentes.

9. Flexbox y Grid en componentes web

Para componentes como tarjetas, menús o grupos de botones, primero hay que mirar cómo están relacionados los elementos.

Para recordar

- **flex-wrap:** permite que los elementos pasen a otra línea.
- **flex-grow:** permite que un elemento ocupe más espacio.
- **flex-shrink:** controla cuánto puede reducirse un elemento.

10. Flexbox vs CSS Grid

No se trata de cuál es mejor, sino de cuál sirve mejor para cada situación.

Si estoy organizando principalmente una fila o una columna, normalmente pienso en Flexbox. Si necesito controlar filas y columnas, pienso en Grid.

Para recordar

- **flex-basis:** define el tamaño base de un elemento.
- **grid-template-areas:** permite organizar el layout mediante áreas con nombres.

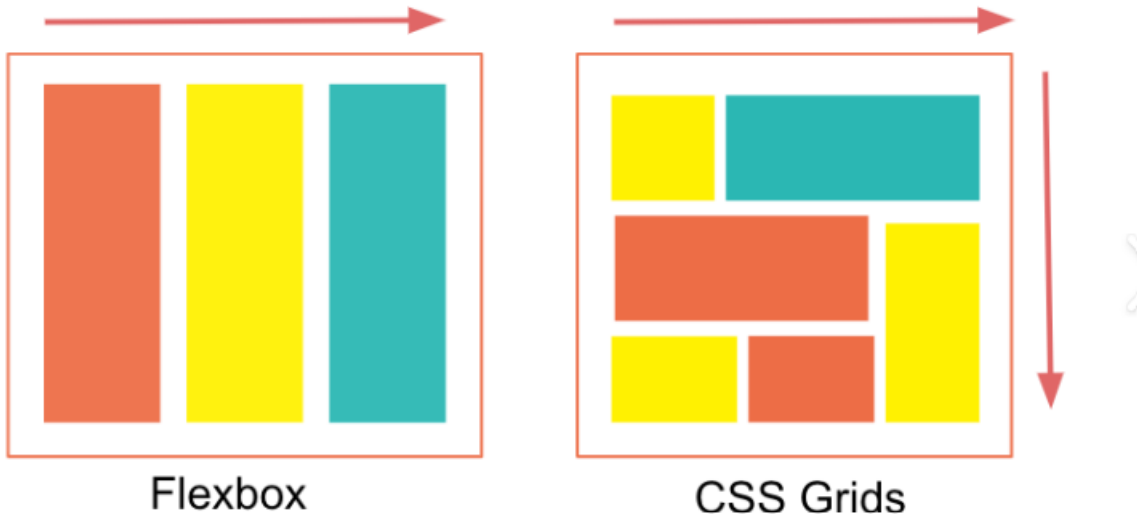
11. Cuándo usar Flexbox y Grid

Antes de escribir CSS conviene mirar cómo está organizado el diseño.

Para recordar

- **flex-direction:** define la dirección principal de Flexbox.
- **grid-template-columns:** define cómo se dividen las columnas de Grid.

Captura: ejemplo donde se vea cuándo conviene utilizar Flexbox y cuándo Grid.



12. Layouts modernos y responsive

Un diseño responsive debe adaptarse al espacio disponible. Grid tiene varias funciones que ayudan a conseguirlo sin poner tantos tamaños fijos.

Para recordar

- **minmax():** define un tamaño mínimo y uno máximo.
- **repeat():** repite columnas o filas.
- **auto-fit:** ajusta automáticamente las columnas que caben.
- **auto-fill:** mantiene las pistas que pueden entrar en el espacio disponible.



13. Patrones de diseño con CSS Grid

Los patrones de Grid sirven como una base para construir diferentes tipos de páginas y después adaptarlos a cada diseño.

Para recordar

- **grid-template-areas:** permite organizar visualmente las áreas.
- **grid-column:** indica dónde empieza o termina un elemento en las columnas.
- **grid-row:** indica dónde empieza o termina un elemento en las filas.



14. Cinco técnicas sencillas de CSS moderno

La idea es aprovechar las funciones actuales de CSS para hacer diseños más flexibles sin llenar el código de ajustes.

Para recordar

- **clamp():** establece un mínimo, un valor flexible y un máximo.
- **aspect-ratio:** mantiene una proporción entre ancho y alto.
- **object-fit:** controla cómo se adapta una imagen dentro de su caja.

15. Layouts CSS con pocas líneas

Grid puede simplificar bastante un diseño usando funciones que permiten que el navegador distribuya el espacio automáticamente.

Para recordar

- **repeat():** evita repetir manualmente las mismas columnas.
- **minmax():** establece un mínimo y un máximo.
- **auto-fit:** adapta las columnas al espacio disponible.

16. Habilidades visuales en diseño web

No todo es código. También hay que fijarse en cómo se ve y se entiende una interfaz: tamaños, espacios, contraste y jerarquía.

Para recordar

- **line-height:** controla el espacio entre líneas.
- **font-size:** define el tamaño del texto.
- **margin:** controla el espacio exterior.
- **padding:** controla el espacio interior.

17. Design Systems

Un Design System sirve para mantener una misma lógica visual en toda una interfaz. Así los componentes no terminan teniendo estilos completamente diferentes.

Para recordar

- **Variables CSS:** guardan valores que se pueden reutilizar.
- **--nombre:** forma de declarar una variable personalizada.
- **var():** permite utilizar una variable.

18. Principios de diseño UI/UX

Una interfaz no debe ser solamente bonita. También debe ser clara, fácil de usar y consistente.

Para recordar

- **contrast:** ayuda a diferenciar elementos y mejorar la lectura.
- **cursor:** indica el tipo de puntero y puede mostrar que algo es interactivo.

- **transition:** hace más suaves algunos cambios visuales.

19. Diseño de Y-frames para reproductor de música mobile

Antes de programar una interfaz es útil pensar primero en su estructura y en la posición de los elementos.

Para recordar

- **width:** define el ancho.
- **height:** define el alto.
- **border-radius:** redondea las esquinas.
- **box-shadow:** agrega sombras y profundidad.

20. Figma y Auto Layout

Auto Layout permite organizar elementos dentro de un frame siguiendo reglas de dirección, espacio y tamaño. Es útil porque los elementos no quedan simplemente puestos a mano.

Para recordar

- **Auto Layout:** organiza y distribuye elementos automáticamente.
- **spacing:** define el espacio entre elementos.
- **padding:** define el espacio interno.

21. Neomorfismo en Figma

El neomorfismo utiliza principalmente luces y sombras suaves para dar sensación de profundidad.

Para recordar

- **box-shadow:** crea las sombras.
- **border-radius:** redondea las esquinas.

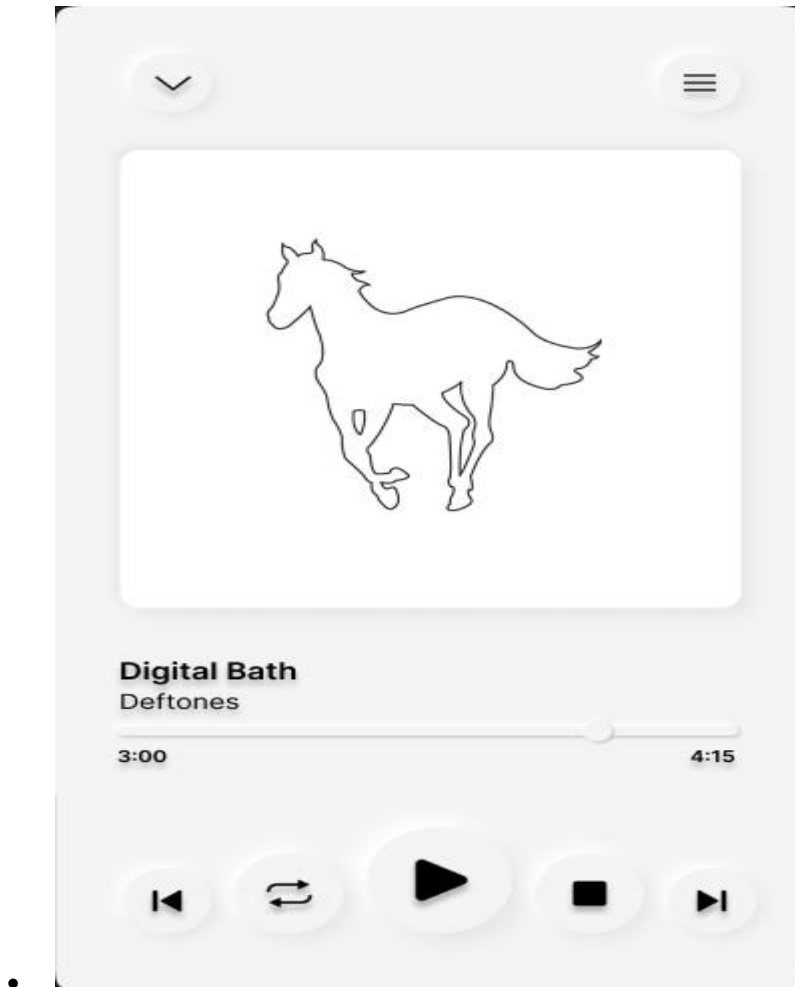
22. Pasar una estructura de Figma a HTML y CSS

La idea es tomar un diseño visual y convertirlo en una estructura real de HTML y CSS.

Primero identifico las secciones y componentes y después pienso cómo distribuirlos.

Para recordar

- **display:** define el comportamiento del elemento.
- **box-sizing:** controla cómo se calcula el tamaño de la caja.
- **max-width:** limita el ancho máximo.



23. Neomorfismo con HTML y CSS

El objetivo es llevar el efecto visual del neomorfismo de Figma al código sin afectar la legibilidad o la facilidad de uso.

Para recordar

- **box-shadow:** define las sombras.
- **background:** define el fondo.
- **border:** puede ayudar a separar elementos.

24. CSS y su evolución

CSS sigue evolucionando y aparecen nuevas características. Por eso es importante revisar qué soportan actualmente los navegadores.

Para recordar

- **@supports:** permite comprobar si el navegador soporta una característica.
- **@media:** permite aplicar estilos dependiendo de condiciones como el tamaño de pantalla.

25. CSS Subgrid

subgrid sirve cuando un elemento necesita seguir las mismas filas o columnas que tiene definido su Grid padre. Esto ayuda bastante cuando varios componentes deben quedar alineados.

Para recordar

- **subgrid:** permite utilizar las pistas del Grid padre.
- **grid-template-columns:** define las columnas.
- **grid-template-rows:** define las filas.

CSS Subgrid

Inheriting Parent Grid Lines

```
.subgrid {  
  display: grid;  
  grid-template-columns: subgrid;  
  grid-template-rows: subgrid;  
}
```

Parent Grid

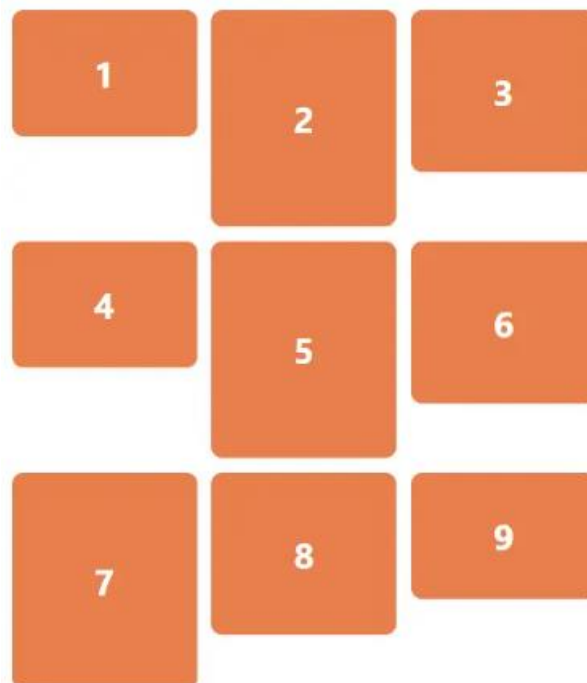


26. Diseño Masonry en CSS

Masonry busca organizar elementos que tienen diferentes alturas aprovechando mejor el espacio disponible y reduciendo huecos.

Para recordar

- **masonry:** permite una distribución tipo mampostería cuando está disponible.
- **columns:** puede utilizarse para organizar contenido en columnas.



27. Consultas de características con @supports

@supports sirve para comprobar si el navegador entiende una característica CSS antes de utilizarla. También permite crear una alternativa cuando no existe soporte.

Para recordar

- **@supports:** comprueba el soporte de una característica.
- **not:** permite comprobar que una condición no se cumple.

28. Mantenerse actualizado en CSS

CSS sigue cambiando y aparecen nuevas propiedades y funciones. No basta con memorizar; también conviene probarlas y revisar su compatibilidad.

Para recordar

- **Can I Use:** sirve para revisar la compatibilidad de una característica entre navegadores.
- **MDN:** sirve para consultar sintaxis, funcionamiento y compatibilidad.